

Git & GitHub 实战笔记

从零开始推送本地仓库到远程

学习笔记

2026 年 3 月 21 日

目录

1	引言	3
2	准备工作：初始化本地 Git 仓库	3
3	关联远程仓库	3
3.1	生动比喻：两种连接方式	3
4	遇到的第一个障碍：SSH 认证失败	4
4.1	原因分析	4
4.2	解决步骤	4
4.2.1	1. 检查是否有 SSH 密钥	4
4.2.2	2. 生成新 SSH 密钥（如果没有）	4
4.2.3	3. 将私钥添加到 SSH 代理	4
4.2.4	4. 将公钥添加到 GitHub	4
4.2.5	5. 测试连接	4
4.3	生动比喻：配钥匙	5
5	第二个障碍：仓库不匹配	5
5.1	原因分析	5
6	第三个障碍：远程仓库有内容，推送被拒绝	5
6.1	生动比喻：两个人同时修改同一份文件	5
6.2	解决方案：拉取合并	6
6.2.1	参数解释	6
6.2.2	合并后的推送	6
6.3	如果希望使用 SSH 而非 HTTPS	6

7	最终成功的命令序列	6
8	常见问题与解决	7
9	总结	7

1 引言

在日常开发中，我们经常需要将本地的代码或文档同步到 GitHub 上进行备份或协作。这个过程看似简单，但初学者经常会遇到 SSH 密钥认证、远程仓库冲突等问题。本文将通过一个真实的案例，逐步讲解如何将本地 Git 仓库推送到 GitHub，并解决过程中遇到的典型错误。

2 准备工作：初始化本地 Git 仓库

假设我们在 Documents 文件夹下有一些文件（比如 PDF 笔记），想要将它们放入 Git 版本控制并上传到 GitHub。

```
1 cd ~/Documents
2 git add .
3 git commit -m "upload"
```

小知识

git add . 会将当前目录下所有文件添加到暂存区；git commit -m "..." 则创建一个提交记录。此时本地仓库已经建立。

3 关联远程仓库

我们需要告诉本地仓库远程仓库的地址。有两种协议：SSH 和 HTTPS。

```
1 # SSH 方式
2 git remote add origin git@github.com:用户名/仓库名.git
3
4 # HTTPS 方式
5 git remote add origin https://github.com/用户名/仓库名.git
```

3.1 生动比喻：两种连接方式

- SSH：就像你有自己家的钥匙，每次开门不需要输密码，但需要先配好钥匙（SSH 密钥）。
- HTTPS：就像用密码开门，每次推拉都需要输入账号密码（或个人访问令牌）。

4 遇到的第一个障碍：SSH 认证失败

当使用 SSH 方式推送时，出现了以下错误：

```
1 git@github.com: Permission denied (publickey).  
2 fatal: Could not read from remote repository.
```

4.1 原因分析

GitHub 不认识我们电脑的 SSH 公钥，因此拒绝了连接。就像你想进一栋大楼，但保安没有你的指纹记录。

4.2 解决步骤

4.2.1 1. 检查是否有 SSH 密钥

```
1 ls -al ~/.ssh
```

如果看到 `id_ed25519` 和 `id_ed25519.pub` 等文件，说明已经有密钥；否则需要生成。

4.2.2 2. 生成新 SSH 密钥（如果没有）

```
1 ssh-keygen -t ed25519 -C "your-email@example.com"
```

一路回车即可（可以设置密码短语，增加安全性）。

4.2.3 3. 将私钥添加到 SSH 代理

```
1 eval "$(ssh-agent -s)"  
2 ssh-add ~/.ssh/id_ed25519
```

4.2.4 4. 将公钥添加到 GitHub

```
1 cat ~/.ssh/id_ed25519.pub
```

复制输出的全部内容，然后登录 GitHub，进入 Settings → SSH and GPG keys → New SSH key，粘贴并保存。

4.2.5 5. 测试连接

```
1 ssh -T git@github.com
```

如果看到 Hi 用户名！ You've successfully authenticated...，说明认证成功。

4.3 生动比喻：配钥匙

生成 SSH 密钥就像配了一把新钥匙（私钥），并把钥匙的“齿纹”（公钥）交给 GitHub 管理员（添加公钥）。从此以后，你每次用私钥开门，管理员都会验证你的身份。

5 第二个障碍：仓库不匹配

SSH 认证通过后，再次推送时又遇到新错误：

```
1 ERROR: Repository not found.
2 fatal: Could not read from remote repository.
```

5.1 原因分析

远程仓库可能不存在，或者本地关联的地址写错了。检查一下远程仓库 URL：

```
1 git remote -v
```

如果发现地址有误，可以使用 `git remote set-url origin 新地址` 修正。

注意

确保你在 GitHub 上已经创建了同名仓库（这里是 My-Knowledge-Base），并且有访问权限。

6 第三个障碍：远程仓库有内容，推送被拒绝

修正远程地址后，再次推送：

```
1 git push -u origin main
```

出现：

```
1 ! [rejected]          main -> main (fetch first)
2 error: failed to push some refs to ...
3 hint: Updates were rejected because the remote contains work that you
   do not
4 hint: have locally.
```

6.1 生动比喻：两个人同时修改同一份文件

想象你和朋友各自有一份相同的文稿。朋友在 GitHub 上已经修改了一些内容，而你本地还是最初版本。当你试图直接覆盖朋友的修改时，Git 会阻止你，因为这样会丢

失朋友的工作。Git 要求你先“同步”一下，即先拉取远程的修改。

6.2 解决方案：拉取合并

我们需要将远程的改动拉到本地，并与本地内容合并，然后再推送。

```
1 git pull origin main --allow-unrelated-histories --no-rebase
```

6.2.1 参数解释

- `--allow-unrelated-histories`：允许合并没有共同祖先的分支。因为本地仓库是全新的，而远程仓库可能已经有一些文件（比如 README），它们的提交历史不相关，所以需要这个参数。
- `--no-rebase`：使用合并（merge）而不是变基（rebase）。合并会创建一个新的合并提交，保留双方历史。

执行这条命令后，Git 会尝试自动合并。如果遇到冲突，需要手动解决（这里没有冲突，因为文件不同）。合并后，Git 可能会打开一个编辑器让你输入合并提交信息，保存后退出即可。

6.2.2 合并后的推送

```
1 git push -u origin main
```

现在应该可以成功推送了。

6.3 如果希望使用 SSH 而非 HTTPS

可以再次修改远程地址：

```
1 git remote set-url origin git@github.com:用户名/仓库名.git
```

然后重新执行拉取和推送。

7 最终成功的命令序列

以下是一个完整的、可复现的命令序列（假设从零开始）：

```
1 # 1. 初始化并提交本地文件
2 cd ~/Documents
3 git init
4 git add .
```

```
5 git commit -m "initial commit"
6
7 # 2. 关联远程仓库 (SSH 方式, 需要先配置好 SSH 密钥)
8 git remote add origin git@github.com:maze01001000/My-Knowledge-Base.git
9
10 # 3. 拉取远程内容并合并 (允许无关历史)
11 git pull origin main --allow-unrelated-histories --no-rebase
12
13 # 4. 推送到远程
14 git push -u origin main
```

8 常见问题与解决

Q1: 问: 为什么第一次 `git pull` 会提示 “需要指定如何协调分歧分支”?

Q2: 答: 因为 Git 2.0 以后默认不自动选择合并策略, 需要显式指定 `--no-rebase` 或 `--rebase`。我们选择 `--no-rebase` 进行合并。

Q3: 问: 如果合并时出现冲突怎么办?

Q4: 答: Git 会在冲突文件中标记 `<<<<<<` 和 `>>>>>>`, 你需要手动编辑文件, 保留想要的内容, 然后 `git add` 并 `git commit` 完成合并。

Q5: 问: 我可以直接强制推送覆盖远程吗?

Q6: 答: 可以, 但非常危险: `git push -f origin main` 会覆盖远程仓库, 导致他人或你自己之前的工作丢失。只有在你确定远程内容不重要时才使用。

9 总结

通过这次实战, 我们学习了:

- 如何初始化本地 Git 仓库并提交文件。
- SSH 密钥的生成、配置和测试。
- 处理远程仓库已存在内容时的合并策略。
- 使用 `--allow-unrelated-histories` 合并独立分支。

Git 的设计哲学是“保护你的工作不被意外覆盖”，虽然初次使用会有些曲折，但一旦理解了其背后的思想，就能游刃有余地管理代码版本。

Happy Coding!